



Програмирани нумерички методи

Директни и итеративни пристапи кон линеарни системи

„Точноста и стабилноста на нумеричките алгоритми се клучни фактори при нивната примена, бидејќи малите грешки во пресметките може да доведат до значајни отстапувања во конечните резултати“

Март, 2026

УНИВЕРЗИТЕТ „ГОЦЕ ДЕЛЧЕВ”- ШТИП
ФАКУЛТЕТ ЗА ИНФОРМАТИКА
Компјутерско инженерство и технологии

СЕМИНАРСКА РАБОТА
Програмирани нумерички методи



Ментор,
Проф-Др. Владо Гичев

Студент,
Мерсие Адемова
број на индекс:102856

Штип, март 2026

Содржина

1. Вовед.....	3
2. Формална спецификација на нумеричкиот модел.....	4
2.1. Услов за постоење и единственост на решение	5
2.2. Класификација на методи	5
2.3. Услов за применливост на итеративни методи	5
2.4. Дефиниција на норма.....	6
2.5. Мерка на грешка	6
2.6. Критериум за запирање	7
2.7. Ограничување на број на итерации	7
2.8. Формална дефиниција на нумеричкиот процес	8
3. Теоретска анализа на алгоритмите и матричните алатки.....	8
4. Практична анализа на тестирањето	12
5. Проширена тест анализа	15
6. Графички кориснички интерфејс и визуелизација на резултатите	20
7. Тест пример и практична валидација на програмата.....	25
7.1. Формулација на тест примерот	26
7.2. Решавање на системот со Гаусова елиминација	27
7.3. Решавање на системот со методот на Јакоби	29
7.4. Решавање на системот со методот на Гаус-Зајдел	31
7.5. Норма на векторот на грешка	34
7.6. Анализа на конвергенција помеѓу последователни итерации	35
7.7. Анализа на грешката во однос на точното решение.....	37
7.8. Графичка анализа на конвергенцијата на решенијата	39
8. Практична верификација на програмската имплементација	41
9. Заклучок	44
10. Користена литература	45

Програмирани нумерички методи

Директни и итеративни пристапи кон линеарни системи

1. Вовед

Во современата примена на математиката, решавањето на систем линеарни равенки претставува една од фундаменталните и најчесто користените задачи. Голем број проблеми од инженерството, физиката, економијата и информатиката можат да се сведат на модел од облик $Ax = b$, каде матрицата A ги претставува меѓусебните односи во системот, векторот x ги содржи непознатите величини, а векторот b ги претставува надворешните влијанија или познатите параметри. Иако формално ваквиот запис е компактен и едноставен, неговото практично решавање во реални услови отвора низа нумерички и теориски прашања поврзани со точноста, стабилноста и ефикасноста на избраниот метод.

За разлика од аналитичките решенија кои се изведуваат симболички и често претставуваат идеални услови, нумеричките методи работат со конечна претизност и апроксимации. Компјутерските пресметки се ограничени од начинот на претставување на реалните броеви, што значи дека секој резултат содржи одредено отстапување од точната математичка вредност. Поради тоа, во нумеричката анализа централно место зазема поимот грешка. Наместо да се третира како недостаток, грешката се разгледува како мерлива и контролирана величина која овозможува проценка на квалитетот на добиеното решение. Разбирањето на природата на грешката е неопходно за правилна интерпретација на резултатите и за избор на соодветен алгоритам.

Методите за решавање на линеарни системи можат да се поделат на директни и итеративни. Директните методи, како што е Гаусовата елиминација, настојуваат во конечен број чекори да го трансформираат системот во еквивалентна форма од која решението се добива експлицитно. Тие се концептуално блиски до алгебарските техники изучувани во основната линеарна алгебра и често

обезбедуваат висока точност за системи со умерена димензија. Од друга страна, итеративните методи, како што се методот на Јакоби и методот на Гаус-Зајдел, го градат решението постепено, преку низа последователни апроксимации. Наместо да го бараат решението во еден чекор, тие се потпираат на процес на приближување чија успешност зависи од структурата на матрицата и од поставениот критериум за запирање.

Практичната важност на овие методи произлегува од фактот дека современите техники и научни проблеми често доведуваат до системи со голема димензија, каде класичните аналитички техники се неизводливи. Во анализата на конструкции, електрични мрежи, механички модели, како и во нумеричкото решавање на диференцијални равенки, системот од линеарни равенки претставува меѓучекор без кој не може да се дојде до финалното решение. Поради тоа, изборот на метод не е само формално прашање, туку одлука која директно влијае врз точноста и стабилноста на целокупниот модел.

Целта на оваа семинарска работа е да се направи теоретска и практична анализа на директниот метод на елиминација и на двата класични итеративни методи, со посебен акцент на поимот грешка и на однесувањето на конвергенцијата. Преку нивна имплементација и споредбена анализа ќе се прикаже како апстрактните математички концепти се манифестираат во конкретна нумеричка пресметка, и како теоретските услови за стабилност и конвергенција се одразуваат во практични резултати. На тој начин, решавањето на системи линеарни равенки ќе биде рагледано не само како алгоритамска постапка, туку како процес во кој математичката теорија и компјутерската реализација се неразделно поврзани.

2. Формална спецификација на нумеричкиот модел

Во рамките на оваа семинарска работа се разгледува нумеричко решавање на линеарен систем од алгебарски равенки во матрична форма:

$$Ax = b$$

Каде што $A \in R^{n \times n}$ е квадратна матрица на коефициенти, $x \in R^n$ е вектор на непознати, а $b \in R^n$ е вектор на познати членови. Димензијата n е конечен природен број.

2.1. Услов за постоење и единственост на решение

Се претпоставува дека матрицата A е несингуларна, односно

$$\det(A) \neq 0$$

Што имплицира постоење на единственост на решението x . Доколку $\det(A) = 0$, системот се смета за сингуларен и не се гарантира единствено решение; во таков случај директниот метод мора да сигнализира неуспех.

2.2. Класификација на методи

Во моделот се разгледуваат два типа нумерички пристапи:

- Директен метод (Гаусова елиминација со парцијално пивотирање),
- Итеративни методи (метод на Јакоби и метод на Гаус-Зајдел).

Кај директниот метод решението се добива по конечен број операции. Кај итеративните методи се конструира низа од апроксимации

$$\{x^{(k)}\}, k = 0, \dots, \infty$$

Каде што $x^{(0)} \in R^n$ е почетна апроксимација, а секој нареден вектор се добива преку дефинирана итеративна шема.

2.3. Услов за применливост на итеративни методи

За да се обезбеди конвергенција на итеративните методи, се воведува услов на строга дијагонална доминантност. Матрицата $A = [a_{ij}]$ е строго дијагонално доминантна ако за секое $i \in \{1, \dots, n\}$ важи:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|$$

Овој услов претставува доволен услов за конвергенција на Jacobi и Gauss-Seidel методите. Доколку матрицата не го задоволува овој услов, конвергенцијата не е гарантирана и алгоритмот може да не се стабилизира.

2.4. Дефиниција на норма

За мерење на големината на векторите во итеративниот процес се користи концептот векторска норма, која претставува функција што на секој вектор му доделува не-негативна реална вредност што ја опишува неговата големина.

Во имплементацијата се користат две најчесто применувани норми.

Бесконечна норма е дефинирана како:

$$\|\vartheta\|_{\infty} = \max_{1 \leq i \leq n} |\vartheta_i|$$

Оваа норма ја мери максималната апсолутна компонента на векторот и обезбедува строг критериум за следење на стабилизацијата на решението, бидејќи ја контролира максималната девијација на било која компонента.

Покрај неа, се користи и Евклидовата норма (норма 2), дефинирана со:

$$\|\vartheta\|_2 = \sqrt{\sum_{i=1}^n \vartheta_i^2}$$

Оваа норма ја мери вкупната геометриска големина на векторот, односно неговата должина во Евклидов простор. Таа обезбедува порамномерна проценка на грешката бидејќи ги зема предвид сите компоненти на векторот.

2.5. Мерка на грешка

Бидејќи точното решение x во практични услови не е однапред познато, се дефинира апсолутна итеративна грешка како разлика меѓу две последователни апроксимации:

$$e^{(k)} = \|x^{(k)} - x^{(k-1)}\|$$

Во зависност од избраната норма, грешката може да се пресметува на два начини.

Кога се користи бесконечната норма, грешката се дефинира како:

$$e^{(k)} = \|x^{(k)} - x^{(k-1)}\|_{\infty}$$

Додека при користење на Евклидовата норма:

$$e^{(k)} = \|x^{(k)} - x^{(k-1)}\|_2$$

Оваа мерка претставува индикатор за динамиката на конвергенција и дозволува практична проценка на стабилноста на итеративниот процес. Како што вредноста на грешката се намалува со зголемување на бројот на итерации, решението се приближува кон стабилна апроксимација на вистинското решение на системот.

2.6. Критериум за запирање

Итеративниот процес се прекинува кога е исполнет условот:

$$e^{(k)} < \varepsilon$$

Каде што $\varepsilon > 0$ е однапред зададена толеранција. Толеранцијата претставува параметар на точност кој го дефинира прифатливото ниво на нумеричка грешка.

2.7. Ограничување на број на итерации

Со цел да се гарантира алгоритамска конечност, се воведува дополнителен услов

$$k \leq \text{max_iterations}$$

Каде што $\text{max_iterations} \in \mathbb{N}$ е однапред зададена константа, типично 500 или 1000. Доколку се достигне оваа граница без исполнување на критериумот за точност, процесот се прекинува и методот се смета за неконвергентен во рамките на зададените параметри.

2.8. Формална дефиниција на нумеричкиот процес

Нумеричкиот модел е дефиниран како пресликување:

$$F: (A, b, x^{(0)}, \varepsilon, \text{max_iterations}) \rightarrow (x^{(k)}, k, e^{(k)}, \text{status})$$

Каде што излезот содржи:

- Финална апроксимација $x^{(k)}$;
- Број на извршени итерации;
- Последна вредност на грешката;
- Статус (конвергенција или дивергенција).

Со оваа спецификација се дефинираат сите математички предуслови, ограничувања и критериуми кои ќе ја управуваат имплементацијата на алгоритмите. Овој дел претставува теоретска основа врз која се темели понатамошната алгоритамска реализација и експериментална анализа.

3. Теоретска анализа на алгоритмите и матричните алатки

Во проектот се користат четири основни компоненти кои го сочинуваат математичкиот модул за решавање на систем линеарни равенки. Првата група функции се однесува на проверка и подготовка на матрицата и векторот, кои се сместени во скрипта насловена како *matrix_utils*. Овие функции обезбедуваат основни проверки пред да се применат решавачките алгоритми. Проверката за квадратна матрица гарантира дека бројот на равенки е еднаков на бројот на непознати, што е предуслов за применливост на класичните методи. Проверката за сингуларност се базира на детекција на премали *pivot* елементи во процесот на елиминација, што укажува дека системот нема уникатно решение. Дополнително, дијагоналната доминација се проверува за да се процени веројатноста на конвергенција на итеративните методи. Сите овие проверки обезбедуваат стабилност и предвидливост на нумеричките методи.

```

matrix_utils.py > ...
import copy

def is_square(A):
    """Проверка дали матрицата е квадратна."""
    if len(A) == 0:
        return False
    n = len(A)
    return all(len(row) == n for row in A)

def is_singular(A, tol=1e-12):
    """Проверка на сингулярност (ако pivot е премал)."""
    n = len(A)
    temp = copy.deepcopy(A)
    for i in range(n):
        pivot = temp[i][i]
        if abs(pivot) < tol:
            return True
        for j in range(i+1, n):
            factor = temp[j][i] / pivot
            for k in range(i, n):
                temp[j][k] -= factor * temp[i][k]
    return False

def is_diagonally_dominant(A):
    """Проверка на строга дијагонална доминација."""
    n = len(A)
    for i in range(n):
        diag = abs(A[i][i])
        off_diag_sum = sum(abs(A[i][j]) for j in range(n) if j != i)
        if diag <= off_diag_sum:
            return False
    return True

```

1. matrix_utils.py

Следните три компоненти се самите алгоритми за решавање на системи линеарни равенки. **Гаусовата елиминација со парцијално пивотирање** е директен метод

кој користи редукција на матрицата кон горнотриаголна форма со цел едноставно да се изврши *back – substitution*. Парцијалното пивотирање овозможува зголемување на нумеричката стабилност со размена на редови, што ја минимизира грешката која може да се акумулира при работа со мали или близу нула елементи на дијагонала.

```
gaussian.py > gaussian_elimination
import copy

def gaussian_elimination(A, b, tol=1e-12):
    """Gaussian elimination with partial pivoting."""
    n = len(A)
    A = copy.deepcopy(A)
    b = copy.deepcopy(b)

    for i in range(n):
        # Partial pivoting
        max_row = max(range(i, n), key=lambda r: abs(A[r][i]))
        if abs(A[max_row][i]) < tol:
            return None, "singular"
        if max_row != i:
            A[i], A[max_row] = A[max_row], A[i]
            b[i], b[max_row] = b[max_row], b[i]

        # Elimination
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(i, n):
                A[j][k] -= factor * A[i][k]
            b[j] -= factor * b[i]

    # Back substitution
    x = [0 for _ in range(n)]
    for i in range(n-1, -1, -1):
        sum_ax = sum(A[i][j] * x[j] for j in range(i+1, n))
        x[i] = (b[i] - sum_ax) / A[i][i]

    return x, "success"
```

2. gaussian.py

Итеративните методи, методот на Јакоби и методот на Гаус-Зајдел, се засноваат на приближување на решението преку последователни итерации. Методот на Јакоби користи копија на претходната итерација за пресметка на секоја компонента од решението, што го прави алгоритмот независен за секоја променлива во тековната итерација, но генерално побавен.

```
jacobi.py > jacobi
def jacobi(A, b, x0=None, epsilon=1e-8, max_iter=500):
    """Jacobi iterative method."""
    n = len(A)
    if x0 is None:
        x0 = [0.0 for _ in range(n)]
    x_old = x0[:]
    iterations_list = [x_old[:]]
    errors_list = []

    for _ in range(max_iter):
        x_new = [0.0 for _ in range(n)]
        for i in range(n):
            s = sum(A[i][j] * x_old[j] for j in range(n) if j != i)
            x_new[i] = (b[i] - s) / A[i][i]

        # Calculate infinity norm error
        error = max(abs(x_new[i] - x_old[i]) for i in range(n))
        errors_list.append(error)
        iterations_list.append(x_new[:])

        if error < epsilon:
            return x_new, iterations_list, errors_list

        x_old = x_new

    return x_new, iterations_list, errors_list
```

3. jacobi.py -прва верзија

Методот на Гаус-Зајдел користи in-place update, односно новите вредности на компонентите веднаш се користат во пресметките за следните непознати, што го забрзува конвергирањето. За двата методи, пресметката на грешка се врши преку бесконечна норма на разликата меѓу последователните решенија, а критериумот

за запирање или на постигната минимална грешка, или на достигнат максимален број итерации.

```
gauss_seidel.py > gauss_seidel
def gauss_seidel(A, b, x0=None, epsilon=1e-8, max_iter=500):
    """Gauss-Seidel iterative method (in-place update)."""
    n = len(A)
    if x0 is None:
        x0 = [0.0 for _ in range(n)]
    x = x0[:]
    iterations_list = [x[:]]
    errors_list = []

    for _ in range(max_iter):
        x_old = x[:]
        for i in range(n):
            s1 = sum(A[i][j] * x[j] for j in range(i))
            s2 = sum(A[i][j] * x_old[j] for j in range(i+1, n))
            x[i] = (b[i] - s1 - s2) / A[i][i]

        # Calculate infinity norm error
        error = max(abs(x[i] - x_old[i]) for i in range(n))
        errors_list.append(error)
        iterations_list.append(x[:])

        if error < epsilon:
            return x, iterations_list, errors_list

    return x, iterations_list, errors_list
```

4. *gauss_seidel.py* - прва верзија

Оваа теоретска рамка овозможува разбирање на математичката логика зад секој алгоритам и улогата на понатамошните проверки и го подготвува теренот за анализа на добиените резултати од тестирањето.

4. Практична анализа на тестирањето

Претходно опишаните модули, во проектот се сместени во заеднички директориум, наречен *core*. За да се потврди исправноста на овој модул и

функционалноста на имплементираните алгоритми, беа извршени серии на тестови со системи линеарни равенки со различни карактеристики. Прво беа тестирани мали системи 2×2 и 3×3 , кои служат како базични примери за проверка на точноста на решенијата. За секој систем беа користени сите три методи: Гаусова елиминација, методот на Јакоби и методот на Гаус-Зајдел.

```
test_core.py > ...
from gaussian import gaussian_elimination
from jacobi import jacobi
from gauss_seidel import gauss_seidel
from matrix_utils import is_diagonally_dominant, is_square, is_singular
# --- Пример 1: 2x2 систем ---
A1 = [[4, 1],
      [2, 3]]
b1 = [1, 2]
print("---- 2x2 систем ----")
# Gaussian
x_gauss, status = gaussian_elimination(A1, b1)
print("Gaussian:", x_gauss, "Status:", status)
# Jacobi
x_jacobi, iter_jacobi, err_jacobi = jacobi(A1, b1, epsilon=1e-10)
print("Jacobi:", x_jacobi)
print("Iterations:", len(iter_jacobi)-1, "Final error:", err_jacobi[-1])
# Gauss-Seidel
x_gs, iter_gs, err_gs = gauss_seidel(A1, b1, epsilon=1e-10)
print("Gauss-Seidel:", x_gs)
print("Iterations:", len(iter_gs)-1, "Final error:", err_gs[-1])
# --- Пример 2: 3x3 систем (дијагонално доминантен) ---
A2 = [[10, -1, 2],
      [-1, 11, -1],
      [2, -1, 10]]
b2 = [6, 25, -11]
print("\n---- 3x3 систем ----")
# Проверка на дијагонална доминација
print("Diagonally dominant:", is_diagonally_dominant(A2))
# Gaussian
x_gauss, status = gaussian_elimination(A2, b2)
print("Gaussian:", x_gauss, "Status:", status)
# Jacobi
x_jacobi, iter_jacobi, err_jacobi = jacobi(A2, b2, epsilon=1e-10)
print("Jacobi:", x_jacobi)
print("Iterations:", len(iter_jacobi)-1, "Final error:", err_jacobi[-1])
# Gauss-Seidel
x_gs, iter_gs, err_gs = gauss_seidel(A2, b2, epsilon=1e-10)
print("Gauss-Seidel:", x_gs)
print("Iterations:", len(iter_gs)-1, "Final error:", err_gs[-1])
```

Резултите покажаа дека Гаусовата елиминација секогаш ја враќа точната вредност на решението и статус “success”, што ја потврдува неговата стабилност и директна апликативност за сите квадратни системи со уникатно решение. Иако Гаусовата елиминација не е итеративен метод, тој служи како референтна точка за споредба на итеративните методи.

Итеративните методи, методот на Јакоби и методот на Гаус-Зајдел, беа тестирани за да се испита нивното конвергирање и влијанието на дијагоналната доминација. Во случајот на 2×2 системот, методот на Јакоби конвергира побавно, барајќи 26 итерации, додека методот на Гаус-Зајдел конвергира за 14 итерации. За 3×3 дијагонално доминантен систем, методот на Јакоби побара 19 итерации, а методот на Гаус-Зајдел само 10. Овие резултати ја потврдуваат познатата тенденција дека методот на Гаус-Зајдел конвергира побрзо поради користење на добиените резултати во тековната итерација.

```
PS E:\UGD\3_godina\semestar6\NumerickiMetodi\ProgammedNumericalMethods\core> python test_core.py
---- 2x2 систем ----
Gaussian: [0.1, 0.6] Status: success
Jacobi: [0.09999999999234344, 0.5999999999540606]
Iterations: 26 Final error: 7.656564271485422e-11
Gauss-Seidel: [0.10000000001148485, 0.5999999999923434]
Iterations: 14 Final error: 5.742420428056505e-11

---- 3x3 систем ----
Diagonally dominant: True
Gaussian: [1.0432692307692306, 2.269230769230769, -1.0817307692307692] Status: success
Jacobi: [1.0432692307598361, 2.269230769237108, -1.0817307692400526]
Iterations: 19 Final error: 4.442557433037564e-11
Gauss-Seidel: [1.043269230769178, 2.269230769230857, -1.0817307692307498]
Iterations: 10 Final error: 4.281019982954604e-12
PS E:\UGD\3_godina\semestar6\NumerickiMetodi\ProgammedNumericalMethods\core> █
```

6. Резултати од *test_core.py*

Дополнително, пресметаните грешки за итеративните методи беа од редот на 10^{-11} до 10^{-12} , што укажува на висока нумеричка стабилност и точност во однос на референтното решение добиено со методот на Гаусова елиминација. Ова е значајно, бидејќи обезбедува доверливост при користење на итеративни методи за поголеми системи каде што директните методи може да бидат прескапи или тешко применливи.

Проверката на *matrix_utils* покажа дека матриците се квадратни, дијагонално доминантни како што е предвидено, и без сингуларност. Ова ја потврдува потребата од овие проверки како предуслов за успешна апликација на итеративните методи и намалување на ризикот од нестабилни резултати.

Заклучно, тестирањето го потврди очекуваното однесување на сите алгоритми. Итеративните методи конвергираат побрзо во системи со дијагонална доминација, Гаусовата елиминација останува точна референтна метода, а *matrix_utils* обезбедува сигурност и предвидливост на нумеричките резултати. Овие резултати го оправдуваат и научниот и практичниот аспект на проектот, бидејќи обезбедуваат стабилно и визуелно разбирливо решение за корисникот.

5. Проширена тест анализа

По иницијалната верификација на алгоритмите врз мали системи од тип 2×2 и 3×3 , се појави потреба од подлабока анализа на нивното однесување при зголемување на димензиите на системот. Основното тестирање ја потврди точноста на имплементацијата, но не даде доволно информации за стабилноста, скалабилноста и динамиката на конвергенција. Токму оваа потреба доведе до проширување на математичкиот модул и креирање на посебна тест инфраструктура.

Со цел да се овозможи системско тестирање на поголеми системи, најнапред беше извршена модификација на итеративните алгоритми. Наместо функциите да враќаат само финално решение, тие беа проширени така што враќаат и целосна листа на меѓурешенија по итерација, како и листа од пресметаните грешки. Оваа надградба овозможува анализа на процесот на конвергенција, а не само на неговиот краен исход. Пресметката на грешката се базира на бесконечна норма на разликата помеѓу две последователни апроксимации, што претставува стабилен и стандарден критериум во нумеричката анализа.

Понатаму, беше стандардизиран интерфејсот на итеративните методи преку воведување на параметрите за толеранција и максимален број на итерации. Ова

овозможи контролирано и повторливо тестирање, при што сите алгоритми се споредуваат под исти услови. Беше дадена и можност за задавање на почетна апроксимација, со што имплементацијата стана поопшта и подготвена за потенцијални понатамошни експерименти.

```
jacobi.py > jacobi
def jacobi(A, b, tol=1e-10, max_iter=1000, x0=None):
    """
    Jacobi iterative method. Returns:solution,list of iteration vectors,list of errors
    """
    n = len(A)
    # иницијална апроксимација
    if x0 is None:
        x_old = [0.0 for _ in range(n)]
    else:
        x_old = x0[:]
    iterations_list = [x_old[:]]
    errors_list = []
    for k in range(max_iter):
        x_new = [0.0 for _ in range(n)]
        for i in range(n):
            s = sum(A[i][j] * x_old[j] for j in range(n) if j != i)
            x_new[i] = (b[i] - s) / A[i][i]
        # infinity norm
        error = max(abs(x_new[i] - x_old[i]) for i in range(n))
        errors_list.append(error)
        iterations_list.append(x_new[:])
        if error < tol:
            return x_new, iterations_list, errors_list
        x_old = x_new
    return x_new, iterations_list, errors_list
```

7. jacobi.py -втора верзија

```

gauss_seidel.py > gauss_seidel
def gauss_seidel(A, b, tol=1e-10, max_iter=1000, x0=None):
    """
    Gauss-Seidel iterative method (in-place update).Returns:solution,
    list of iteration vectors,list of errors
    """
    n = len(A)
    # иницијална апроксимација
    if x0 is None:
        x = [0.0 for _ in range(n)]
    else:
        x = x0[:]
    iterations_list = [x[:]]
    errors_list = []
    for k in range(max_iter):
        x_old = x[:]
        for i in range(n):
            s1 = sum(A[i][j] * x[j] for j in range(i))
            s2 = sum(A[i][j] * x_old[j] for j in range(i + 1, n))
            x[i] = (b[i] - s1 - s2) / A[i][i]
        # infinity norm
        error = max(abs(x[i] - x_old[i]) for i in range(n))
        errors_list.append(error)
        iterations_list.append(x[:])
        if error < tol:
            return x, iterations_list, errors_list
    return x, iterations_list, errors_list

```

8. gauss_seidel.py -втора верзија

За да се избегне рачното конструирање на поголеми системи и да се обезбеди конзистентност во тестирањето, беше имплементирана функција за автоматско генерирање на дијагонално доминантни матрици. Генераторот креира случајни елементи надвор од дијагоналата, а дијагоналните елементи ги поставува така што нивната апсолутна вредност ја надминува сумата на апсолутните вредности на останатите елементи во редот. Со ова се обезбедува контролирана средина за анализа на брзината на конвергенција при различни димензии.

Како директна последица на овие надградби беше креирана посебна скрипта за проширано тестирање. Скриптата автоматски генерира системи, во нашиот случај со диманзии 4×4 , 5×5 и 10×10 и ги применува сите имплементирани методи врз нив. На овој начин тестирањето премина од едноставна функционална проверка на експериментална анализа. Наместо да се проверува само дали алгоритмот враќа решение, се анализира бројот на итерации потребни за конвергенција, финалната грешка и односот помеѓу директниот и итеративните методи.

```
test_large_systems.py > ...
from gaussian import gaussian_elimination
from jacobi import jacobi
from gauss_seidel import gauss_seidel
from matrix_utils import generate_diagonally_dominant_system

def test_system(n):
    print(f"\n----- {n}x{n} систем -----")

    A, b = generate_diagonally_dominant_system(n)

    # Gaussian
    sol_g, status = gaussian_elimination(A, b)
    print("Gaussian status:", status)

    # Jacobi
    sol_j, iter_j, errors_j = jacobi(A, b, tol=1e-10, max_iter=1000)
    print("Jacobi iterations:", len(iter_j))
    print("Jacobi final error:", errors_j[-1])

    # Gauss-Seidel
    sol_gs, iter_gs, errors_gs = gauss_seidel(A, b, tol=1e-10, max_iter=1000)
    print("Gauss-Seidel iterations:", len(iter_gs))
    print("Gauss-Seidel final error:", errors_gs[-1])

if __name__ == "__main__":
    test_system(4)
    test_system(5)
    test_system(10)
```

Резултатите од тестирањето покажаа дека во сите случаи Гаусовата елиминација успешно го пресметува решението, што ја потврдува несингуларноста на генерираните системи. Итеративните методи покажаа стабилна конвергенција со грешка од редот 10^{-11} , што е во согласност со зададената толеранција. Дополнително, методот на Гаус-Зајдел во сите тестирани случаи конвергира побрзо од методот на Јакоби, што повторно ја потврдува теоретската предност на in-place ажурирањето.

```
PS E:\UGD\3_godina\semestar6\NumerickiMetodi\ProgrammedNumericalMethods\core> python test_large_systems.py
>>

---- 4x4 систем ----
Gaussian status: success
Jacobi iterations: 24
Jacobi final error: 2.407181398655922e-11
Gauss-Seidel iterations: 14
Gauss-Seidel final error: 9.419559576784309e-11

---- 5x5 систем ----
Gaussian status: success
Jacobi iterations: 22
Jacobi final error: 6.566161503407386e-11
Gauss-Seidel iterations: 17
Gauss-Seidel final error: 6.482038517052757e-11

---- 10x10 систем ----
Gaussian status: success
Jacobi iterations: 24
Jacobi final error: 3.599925912922686e-11
Gauss-Seidel iterations: 14
Gauss-Seidel final error: 3.298443289334596e-11
```

10. Резултати од тестирањето на поголеми матрици

Интересен аспект на анализата е фактот што бројот на итерации не се зголеми значително со растот на димензијата до 10×10 . Ова укажува дека конвергенцијата е посилно поврзана со степенот на дијагонална доминација отколку со самата големина на системот. Со други зборови, структурата на матрицата има поголемо влијание врз динамиката на конвергенција од нејзината димензија во овој опсег.

Проширената тест анализа ја потврди стабилноста, точноста и скалабилноста на имплементацијата. Истовремено, таа ја оправда потребата од направените надградби, кои овозможуваат не само решавање на системи, туку и системско проучување на нумеричкото однесување на алгоритмите. Ова претставува клучен

чекор кон следната фаза на проектот, во која овие аналитички податоци ќе бидат визуелно претставени и интерпретирани.

6. Графички кориснички интерфејс и визуелизација на резултатите

Досегашните тестирања беа извршени со хардкодирани примери со цел проверка на математичката исправност и точноста на имплементираните методи. Добиените резултати потврдија дека алгоритмите функционираат согласно теоретските очекувања и обезбедуваат точни решенија во зададените услови. Ваквата стабилност и прецизност создаде поволни услови за понатамошно проширување на програмата и нејзино адаптирање за реална корисничка употреба.

За таа цел беше креиран графички интерфејс кој овозможува програмата да се користи интерактивно, без потреба од менување на изворниот код. Преку интерфејсот, корисникот најпрво ја дефинира големината на системот, по што динамички се креира простор за внес на коефициентите и независните членови. Дополнително, може да се зададе толеранција за грешка, што директно влијае врз критериумот за запирање кај итеративните методи.

Покрај овие параметри, интерфејсот овозможува и избор на векторска норма која ќе се користи при пресметката на грешката. Изборот се врши преку dropdown листа, при што корисникот може да избере меѓу Евклидовата норма (норма 2) и бесконечна норма. Ова овозможува споредба на однесувањето на итеративните методи при различни критериуми за мерење на грешката.

По дефинирањето на системот, програмата автоматски генерира форма за внес на равенките, како и полиња за задавање на почетни вредности за итеративните методи. Во овој дел е имплементирано и копче за освежување (Нов систем) кое овозможува задавање на параметри за нов систем без потреба од излегување од апликацијата. Со ова корисникот може брзо да тестира различни почетни

вредности или параметри и да ги споредува добиените резултати, што ја прави програмата погодна за експериментална анализа на конвергенцијата на методите.

Параметри

Број равенки

Број непознати

Толеранција ϵ

Норма

Генерирај систем

x_1 + x_2 + x_3 =

x_1 + x_2 + x_3 =

x_1 + x_2 + x_3 =

Почетно решение $x^{(0)}$

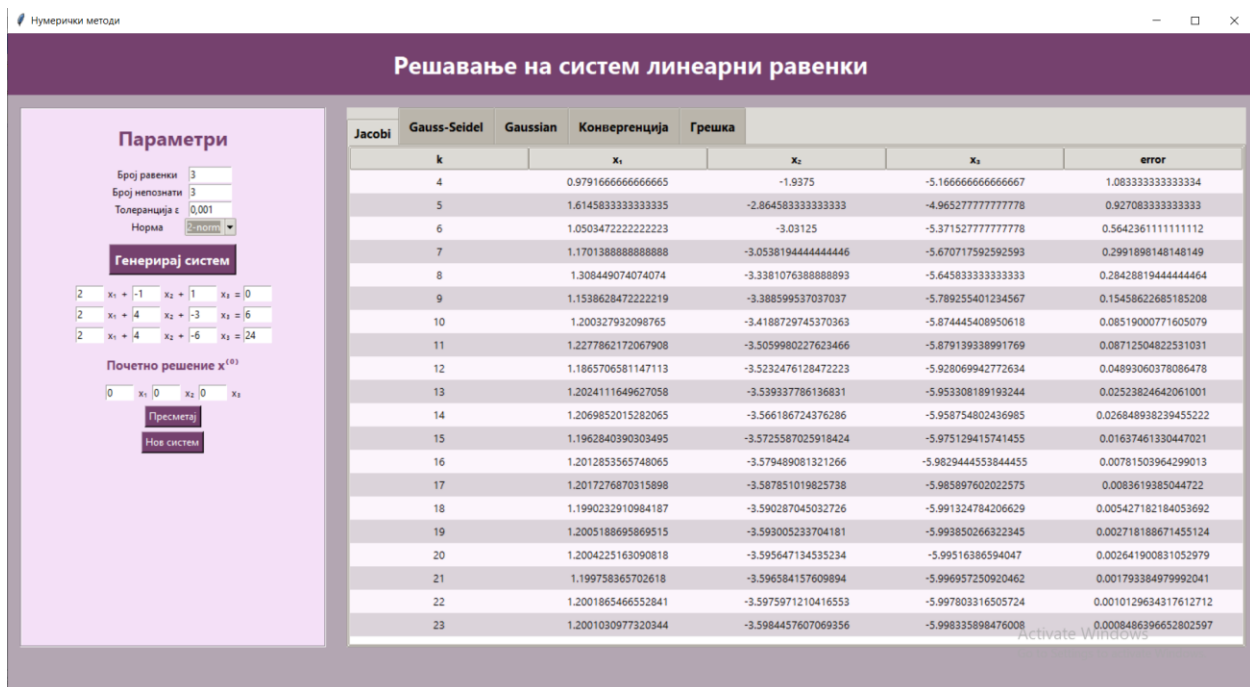
x_1 x_2 x_3

Пресметај

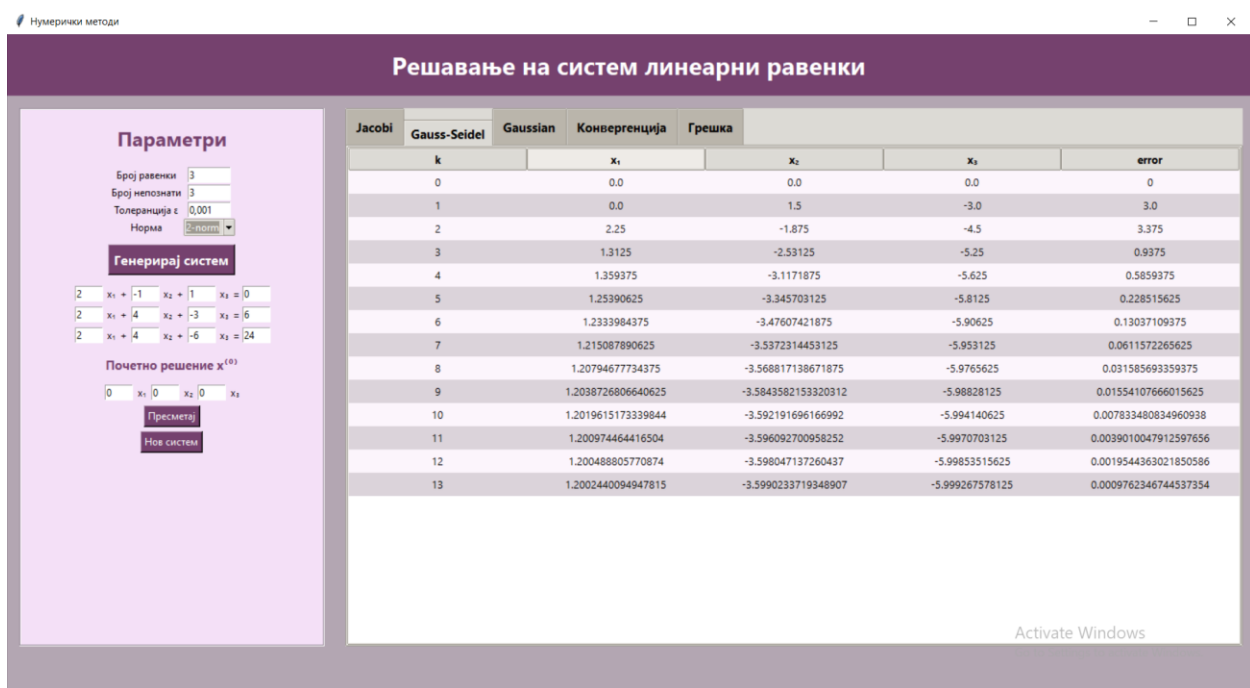
Нов систем

11. Форма за внес на параметри за системот на линеарни равенки

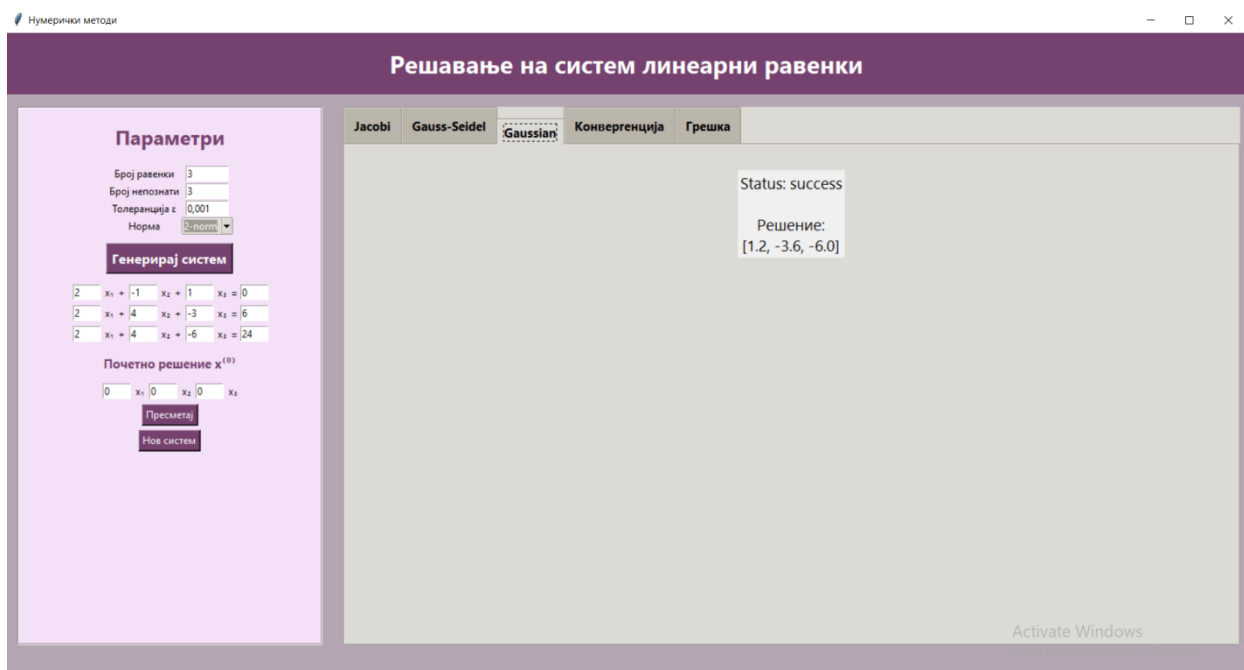
По внесот на параметрите и активирање на пресметката, резултатите се организирани во повеќе прегледни целини. За секој метод се прикажува конечниот резултат, додека кај итеративните методи се прикажуваат сите меѓурекултати и соодветните вредности на грешката. Табеларниот приказ овозможува јасно следење на постепеното приближување кон решението, при што секоја итерација е нумерички претставена и структурирана.



12. Резултати од итеративниот метод на Јакоби



13. Резултати од итеративниот метод на Гаус-Зајдел

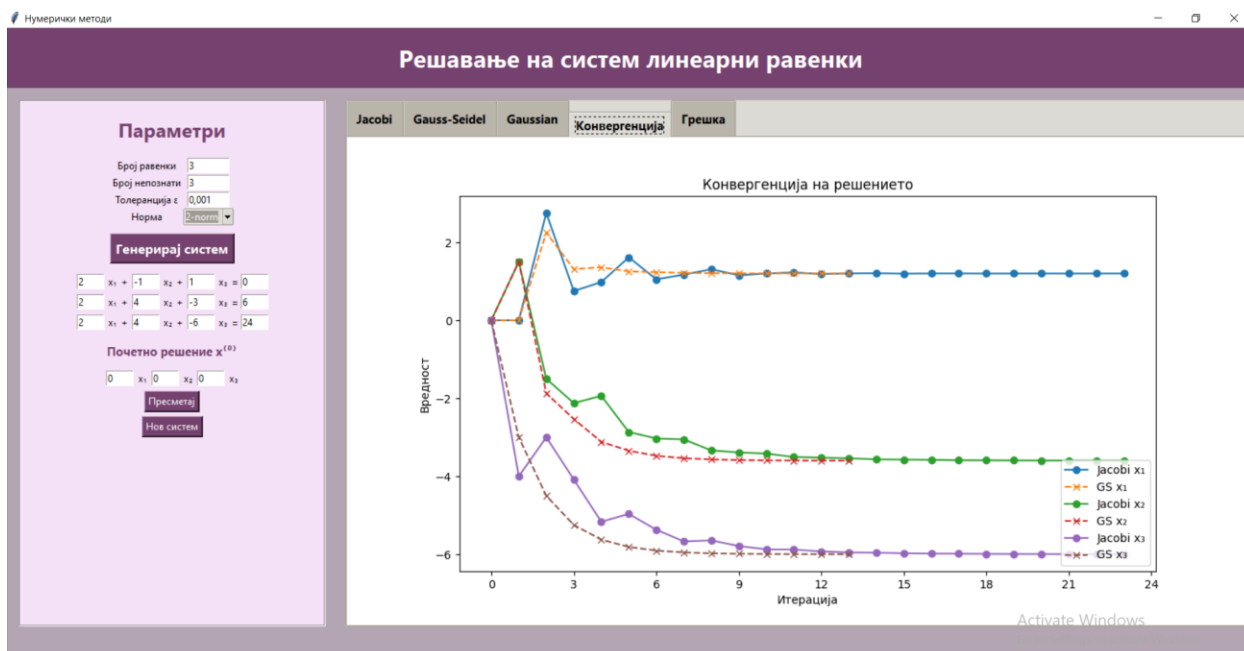


14. Резултат од методот на Гаусова елиминација

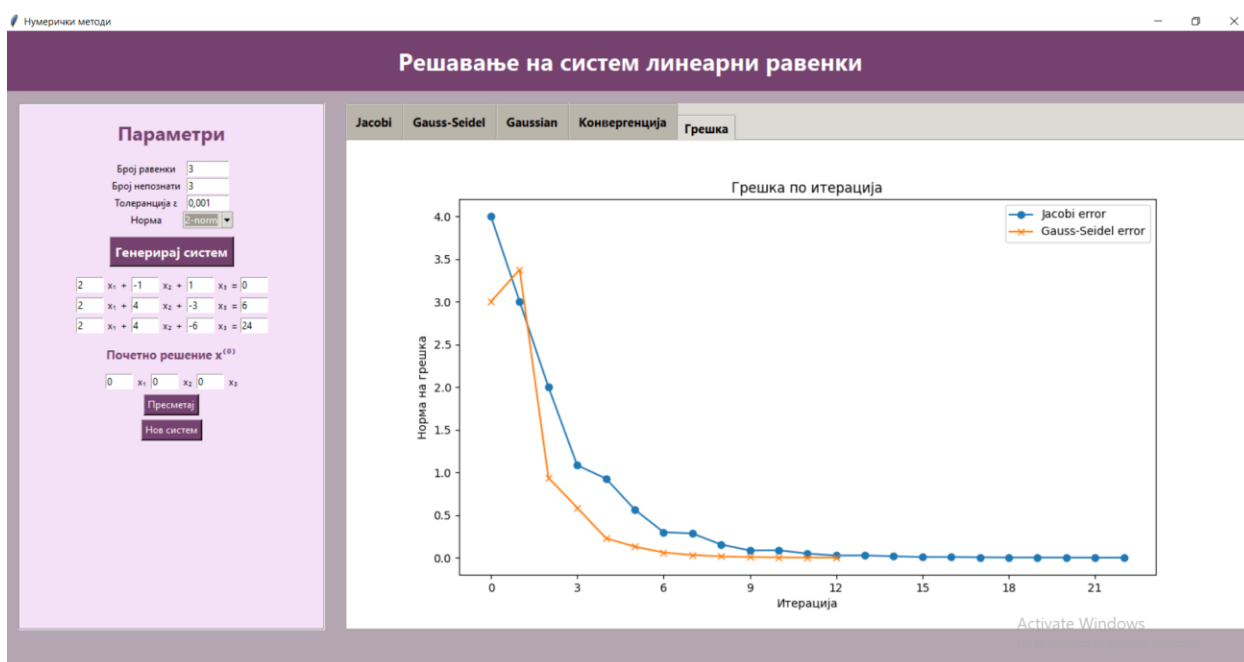
Покрај табеларниот приказ на резултатите, програмата овозможува и графичка визуелизација на процесот на конвергенција. Вредностите на непознатите се прикажуваат по итерации, што овозможува визуелно следење на постепеното стабилизирање на решението.

Во интерфејсот се прикажуваат два посебни графици, секој со различна аналитичка улога.

Првиот график ја прикажува конвергенцијата на непознатите, односно промената на вредностите на компонентите на векторот на решение во текот на итерациите. На оската x е прикажан бројот на итерацијата, додека на оската y се прикажани пресметаните вредности на непознатите. Со овој график може јасно да се види како секоја компонента на решението постепено се приближува кон стабилна вредност.



15. Визуелизација на резултатите од итеративните методи



16. Визуелизација на грешката од итеративните методи

Во случај кога методот ја достигне зададената толеранција, процесот се пркинува и се прикажува конечниот резултат. Доколку не се постигне конвергенција во рамките на дозволеният број итерации, програмата јасно го означува тој исход. На

тој начин, корисникот добива целосна информација не само за решението, туку и за однесувањето на методот во конкретниот случај.

Со ова проширување, програмата преминува од тестна имплементација со фиксни примери во функционална апликација наменета за интерактивна и практична употреба, задржувајќи ја математичката точност потврдена во почетната фаза на тестирањето.

7. Тест пример и практична валидација на програмата

Со цел да се демонстрира практичната примена на имплементираниите нумерички методи, во ова поглавје се разгледува конкретен пример на систем на линеарни равенки. Примерот претставува стандарден тест случај кој овозможува детална анализа на однесувањето на директниот метод на Гаусова елиминација, како и на итеративните методи: методот на Јакоби и методот на Гаус-Зајдел.

Најпрво ќе биде претставен зададениот систем на равенки во неговата аналитичка форма. Потоа системот ќе биде решен рачно, со примена на соодветните нумерички постапки, со што ќе се прикаже текот на пресметките и добиените апроксимации по итерации. Рачното решавање овозможува јасно согледување на чекорите на алгоритмите и служи како референтна основа за верификација на програмската имплементација.

Во последниот дел од поглавјето, истите параметри од тест примерот ќе бидат внесени во развиената програмска апликација. На тој начин ќе се изврши експериментална проверка на резултатите, при што ќе се анализираат табеларните и графичките излези добиени од програмата. Овој пристап овозможува споредба помеѓу рачното решавање и автоматизираната пресметка, со што се потврдува коректноста на имплементираниите алгоритми и нивната практична применливост.

7.1. Формулација на тест примерот

За демонстрација на примената на имплементираните нумерички методи се рагледува следниот систем на три линеарни равенки со три напознати:

$$\begin{cases} -5x_1 + 2x_2 + x_3 = 11 \\ x_1 + 3x_2 - 2x_3 = -4 \\ -2x_1 + x_2 + 4x_3 = 8 \end{cases}$$

Системот може да се запише и во матрична форма:

$$Ax = b$$

каде:

$$A = \begin{bmatrix} -5 & 2 & 1 \\ 1 & 3 & -2 \\ -2 & 1 & 4 \end{bmatrix}, \quad x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}, \quad b = \begin{bmatrix} 11 \\ -4 \\ 8 \end{bmatrix}$$

Овој систем ќе биде решен со три различни нумерички пристапи:

- Директрн метод- Гаусова елиминација
- Итеративен метод на Јакоби
- Итеративен метод на Гаус-Зајдел

Кај итеративните методи ќе се усвои почетна апроксимација:

$$x^{(0)} = (0, 0, 0)$$

И ќе се извршат пет итерации, со цел да се анализира процесот на конвергенција на решението.

По добивањето на решението со Гаусова елиминација, тоа ќе се третира како референтно решение. Потоа ќе се пресмета векторот на грешка, дефиниран како разлика помеѓу точниот резултат и апроксимациите добиени со итеративните методи. За негово квантитативно мерење ќе се користи Евклидовата норма (норма 2).

Дополнително, вредностите на непознатите x_1, x_2, x_3 добиени во секоја итерација ќе бидат прикажани на заеднички график, при што на апсцисата се прикажува бројот на итерации, а на ординатата се прикажуваат вредностите на непознатите.

7.2. Решавање на системот со Гаусова елиминација

Гаусова елиминација е метод од линеарна алгебра кој се користи за решавање системи од линеарни равенки. Методот се базира на редукција на системот со елементарни операции над равенките.

Основната идеја е системот постепено да се претвори во горно-триаголен облик, при што се елиминираат непознатите под главната дијагонала. Ова се прави со три дозволени операции:

- Замена на редови (размена на две равенки);
- Множење на равенка со ненулта константа;
- Собирање на една равенка со друга помножена со константа.

Главните чекори на методот се:

1. Од првата равенка се елиминира првата непозната од сите други равенки;
2. Од втората равенка се елиминира втората непозната од равенките под неа;
3. Процесот продолжува додека системот не стане триаголен систем;
4. Откако системот е во триаголен облик, непознатите се наоѓаат со обратно враќање (back substitution) – прво се наоѓа последната непозната, па претходните.

$$-5x_1 + 2x_2 + x_3 = 11$$

$$x_1 + 3x_2 - 2x_3 = -4 \text{ @ } m_2^{(1)} = -\frac{1}{5}$$

$$-2x_1 + x_2 + 4x_3 = 8 \text{ @ } m_3^{(1)} = \frac{2}{5}$$

$$2) \quad x_1 + 3x_2 - 2x_3 + \frac{1}{5}(-5x_1 + 2x_2 + x_3 - 11) = -4$$

$$3) \quad -2x_1 + x_2 + 4x_3 - \frac{2}{5}(-5x_1 + 2x_2 + x_3 - 11) = 8$$

$$2) \quad \cancel{x_1} + 3x_2 - 2x_3 - \cancel{x_1} + \frac{2x_2}{5} + \frac{x_3}{5} - \frac{11}{5} = -4 / \cdot 5$$

$$3) \quad \cancel{-2x_1} + x_2 + 4x_3 + \cancel{2x_1} - \frac{4x_2}{5} - \frac{2x_3}{5} - \frac{22}{5} = 8 / \cdot 5$$

$$2) \quad 15x_2 - 10x_3 + 2x_2 + x_3 - 11 = -20$$

$$3) \quad 5x_2 + 20x_3 - 4x_2 - 2x_3 + 22 = 40$$

$$2) \quad 17x_2 - 9x_3 = -9$$

$$3) \quad x_2 + 18x_3 = 18$$

$$-5x_1 + 2x_2 + x_3 = 11$$

$$17x_2 - 9x_3 = -9$$

$$x_2 + 18x_3 = 18 \text{ @ } m_3^{(2)} = \frac{1}{17}$$

$$3) \quad x_2 + 18x_3 - \frac{1}{17}(17x_2 - 9x_3 + 9) = 18$$

$$3) \quad x_2 + 18x_3 - x_2 + \frac{9x_3}{17} - \frac{9}{17} = 18 / \cdot 17$$

$$3) \quad 306x_3 + 9x_3 = 9 + 306$$

$$3) \quad 315x_3 = 315$$

$$3) \quad x_3 = \frac{315}{315}$$

$$3) \quad x_3 = 1$$

$$2) \quad 17x_2 - 9x_3 = -9$$

$$2) \quad 17x_2 - 9 \cdot 1 = -9$$

$$2) \quad x_2 = \frac{9-9}{17}$$

$$2) \quad x_2 = 0$$

$$1) \quad -5x_1 + 2x_2 + x_3 = 11$$

$$1) \quad -5x_1 + 2 \cdot 0 + 1 = 11$$

$$1) \quad -5x_1 + 1 = 11$$

$$1) \quad x_1 = -\frac{11-1}{5}$$

1) $x_1 = -2$

$$x_1, x_2, x_3 = (-2, 0, 1)$$

7.3. Решавање на системот со методот на Јакоби

Методот на Јакоби е итеративен нумерички метод што се користи за приближно решавање на системи од линеарни равенки.

Основната идеја е секоја равенка од системот да се реши по една непозната, а потоа со почетна претпоставка за вредностите на непознатите се пресметуваат нови приближни вредности.

Во секоја итерација новите вредности на непознатите се пресметуваат само со вредностите од **претходната итерација**.

Главните чекори на методот се:

1. Системот се преобразува така што секоја равенка е изразена по една непозната;
2. Се избира почетна апроксимација за непознатите;
3. Се пресметуваат нови вредности користејќи ги формулите;
4. Процесот се повторува додека решението не се стабилизира (конвергира).

$$-5x_1 + 2x_2 + x_3 = 11$$

$$x_1 + 3x_2 - 2x_3 = -4$$

$$-2x_1 + x_2 + 4x_3 = 8$$

$$x_1 = \frac{2x_2 + x_3 - 11}{5}$$

$$x_2 = \frac{-4 - x_1 + 2x_3}{3}$$

$$x_3 = \frac{8 + 2x_1 - x_2}{4}$$

Почетно решение: $x^{(0)} = (0, 0, 0)$

Прва итерација:

$$x_1^{(1)} = \frac{2 \cdot 0 + 0 - 11}{5} = -\frac{11}{5} = -2.2$$

$$x_2^{(1)} = \frac{-4 - 0 + 2 \cdot 0}{3} = -\frac{4}{3} = -1.333$$

$$x_3^{(1)} = \frac{8 + 2 \cdot 0 - 0}{4} = \frac{8}{4} = 2$$

$$x^{(1)} = (-2.2, -1.333, 2)$$

Втора итерација:

$$x_1^{(2)} = \frac{2 \cdot (-1.333) + 2 - 11}{5} = \frac{-2.666 - 9}{5} = \frac{-11.666}{5} = -2.333$$

$$x_2^{(2)} = \frac{-4 + 2.2 + 2 \cdot 2}{3} = \frac{2.2}{3} = 0.733$$

$$x_3^{(2)} = \frac{8 + 2 \cdot (-2.2) - (-1.333)}{4} = \frac{8 - 4.4 + 1.333}{4} = \frac{4.933}{4} = 1.233$$

$$x^{(2)} = (-2.333, 0.733, 1.233)$$

Трета итерација:

$$x_1^{(3)} = \frac{2 \cdot (0.733) + 1.233 - 11}{5} = \frac{-8.301}{5} = -1.6602$$

$$x_2^{(3)} = \frac{-4 - (-2.333) + 2 \cdot 1.233}{3} = \frac{-1.667 + 2.466}{3} = \frac{0.799}{3} = 0.266$$

$$x_3^{(3)} = \frac{8 + 2 \cdot (-2.333) - 0.733}{4} = \frac{8 - 4.666 - 0.733}{4} = 0.6502$$

$$x^{(3)} = (-1.6602, 0.266, 0.6502)$$

Четврта итерација:

$$x_1^{(4)} = \frac{2 \cdot 0.266 + 0.6502 - 11}{5} = \frac{-8.301}{5} = -1.963$$

$$x_2^{(4)} = \frac{-4 - (-1.6602) + 2 \cdot 0.6502}{3} = \frac{-2.339 + 1.3004}{3} = -0.3464$$

$$x_3^{(4)} = \frac{8 + 2 \cdot (-1.6602) - 0.266}{4} = \frac{8 - 3.3204 - 0.266}{4} = 1.1034$$

$$x^{(4)} = (-1.963, -0.3464, 1.1034)$$

Петта итерација:

$$x_1^{(5)} = \frac{2 \cdot (-0.6434) + 1.1034 - 11}{5} = \frac{-0.8728 - 9.8966}{5} = -2.1538$$

$$x_2^{(5)} = \frac{-4 - (-1.963) + 2 \cdot 1.1034}{3} = \frac{-2.037 + 2.2068}{3} = 0.0566$$

$$x_3^{(5)} = \frac{8 + 2 \cdot (-1.963) - (-0.3464)}{4} = \frac{8 - 3.926 + 0.3464}{4} = 1.1051$$

$$x^{(5)} = (-2.1538, 0.0566, 1.1051)$$

Табела за методот на Јакоби						
Итерација k	0	1	2	3	4	5
x1	0	-2.2	-2.333	-1.66.2	-1.963	-2.1538
x2	0	-1.333	0.733	0.266	-0.3464	0.0566
x3	0	2	1.233	0.6502	1.1034	1.1051

17. Резултати од меѓу итерациите од методот на Јакоби

7.4. Решавање на системот со методот на Гаус-Зајдел

Методот на Гаус-Зајдел е итеартивен нумерички метод за приближно решавање на систем од линеарни равенки.

Основната идеја е системот од равенки да се преобрази така што секоја равенка е изразена по една непозната, а потоа со почетни претпоставени вредности за непознатите се пресметуваат нови приближни решенија.

За разлика од методот на Јакоби, кај Гаус-Зајдел новодобиените вредности веднаш се користат во **тековната итерација** за пресметување на следните непознати.

Главните чекори на методот се:

1. Системот се изразува така што секоја равенка е решена по една непозната;
2. Се задава почетна апроксимација за непознатите;
3. Се пресметуваат нови вредности редоследно, при што веднаш се користат новите резултати;
4. Итерациите се повторуваат додека разликата меѓу две последователни решенија не стане доволно мала.

$$-5x_1 + 2x_2 + x_3 = 11$$

$$x_1 + 3x_2 - 2x_3 = -4$$

$$-2x_1 + x_2 + 4x_3 = 8$$

$$x_1 = \frac{2x_2 + x_3 - 11}{5}$$

$$x_2 = \frac{-4 - x_1 + 2x_3}{3}$$

$$x_3 = \frac{8 + 2x_1 - x_2}{4}$$

Почетно решение: $x^{(0)} = (0, 0, 0)$

Прва итерација:

$$x_1^{(1)} = \frac{2 \cdot 0 + 0 - 11}{5} = -\frac{11}{5} = -2.2$$

$$x_2^{(1)} = \frac{-4 - (-2.2) + 2 \cdot 0}{3} = \frac{-4 + 2.2 + 0}{3} = \frac{-1.8}{3} = -0.6$$

$$x_3^{(1)} = \frac{8 + 2 \cdot (-2.2) - (-0.6)}{4} = \frac{8 - 4.4 + 0.6}{4} = 1.05$$

$$x^{(1)} = (-2.2, -0.6, 1.05)$$

Втора итерација:

$$x_1^{(2)} = \frac{2 \cdot (-0.6) + 1.05 - 11}{5} = \frac{-1.2 - 9.95}{5} = -2.23$$

$$x_2^{(2)} = \frac{-4 - (-2.23) + 2 \cdot 1.05}{3} = \frac{-1.77 + 2.1}{3} = 0.11$$

$$x_3^{(2)} = \frac{8 + 2 \cdot (-2.23) - 0.11}{4} = \frac{8 - 4.46 - 0.11}{4} = 0.8575$$

$$x^{(2)} = (-2.23, 0.11, 0.8575)$$

Трета итерација:

$$x_1^{(3)} = \frac{2 \cdot 0.11 + 0.8575 - 11}{5} = \frac{-9.9225}{5} = -1.9845$$

$$x_2^{(3)} = \frac{-4 - (-1.9845) + 2 \cdot 0.8575}{3} = \frac{-2.0155 + 1.715}{3} = -0.100$$

$$x_3^{(3)} = \frac{8 + 2 \cdot (-1.9845) - (-0.100)}{4} = \frac{8 - 3.969 + 0.100}{4} = 1.032$$

$$x^{(3)} = (-1.9845, -0.100, 1.032)$$

Четврта итерација:

$$x_1^{(4)} = \frac{2 \cdot (-0.100) + 1.032 - 11}{5} = \frac{-0.2 - 9.968}{5} = -2.0336$$

$$x_2^{(4)} = \frac{-4 - (-2.0336) + 2 \cdot 1.032}{3} = \frac{-1.9664 + 2.064}{3} = 0.0325$$

$$x_3^{(4)} = \frac{8 + 2 \cdot (-2.0336) - 0.0325}{4} = \frac{8 - 4.0672 - 0.0325}{4} = 0.9750$$

$$x^{(4)} = (-2.0336, 0.0325, 0.9750)$$

Петта итерација:

$$x_1^{(5)} = \frac{2 \cdot 0.0325 + 0.9750 - 11}{5} = \frac{-9.96}{5} = -1.992$$

$$x_2^{(5)} = \frac{-4 - (-1.992) + 2 \cdot 0.9750}{3} = \frac{-2.008 + 1.95}{3} = 0.0193$$

$$x_3^{(5)} = \frac{8 + 2 \cdot (-1.992) - (-0.0193)}{4} = \frac{8 - 3.984 + 0.0193}{4} = 1.008$$

$$x^{(5)} = (-1.992, -0.0193, 1.008)$$

Табела за методот на Гаус-Зајдел						
Итерација k	0	1	2	3	4	5
x1	0	-2.2	-2.23	-1.9845	-2.0336	-1.992
x2	0	-0.6	0.11	-0.1	0.0325	0.0193
x3	0	1.05	0.8575	1.032	0.0193	1.008

18. Резултати од меѓу итерациите од методот на Гаус-Зајдел

7.5. Норма на векторот на грешка

Норма е математичка функција која ја мери големината или должината на еден вектор. Таа претставува начин квантитативно да се отцени колку е голем или оддалечен некој вектор.

Во нумеричките методи нормата се користи за мерење на грешката и за следење на конвергенцијата на итеративните алгоритми.

Најчесто користени норми се:

Бесконечна норма:

$$\|\vartheta\|_{\infty} = \max_{1 \leq i \leq n} |\vartheta_i|$$

Таа ја претставува најголемата апсолутна компонента на векторот.

Евклидовата норма (норма 2):

$$\|\vartheta\|_2 = \sqrt{\sum_{i=1}^n \vartheta_i^2}$$

Оваа норма ја претставува геометриската должина на векторот во Евклидов простор.

Векторот на грешка ја претставува разликата помеѓу точното решение на системот и приближното решение добиено со некој нумерички метод.

Ако x е точното решение, а $x^{(k)}$ е приближното решение во k -тата итерација, тогаш векторот на грешка е:

$$e^{(k)} = x - x^{(k)}$$

Големината на оваа грешка се мери со некоја норма:

$$\|e^{(k)}\|$$

Колку што нормата на векторот на грешка е помала, толку приближното решение е поблиску до точниот резултат. Во итеративните методи намалувањето на оваа вредност укажува дека методот конвергира кон вистинското решение.

7.6. Анализа на конвергенција помеѓу последователни итерации

За да се анализира стабилизирањето на решението се пресметува Евклидовата норма на разликата помеѓу две последователни итерации:

$$\|x^{(k+1)} - x^{(k)}\|_2$$

Оваа величина ја мери промената на решението од една во следна итерација.

Евклидовата норма е дефинирана како:

$$\|\vartheta\|_2 = \sqrt{\vartheta_1^2 + \vartheta_2^2 + \dots + \vartheta_n^2}$$

Во контекст на итеративните методи оваа норма претставува критериум за конвергенција.

Ако вредноста $\|x^{(k+1)} - x^{(k)}\|$ се намалува со секоја итерација, тогаш методот конвергира кон стабилно решение.

Метод на Јакоби:

$$\begin{aligned}\|x^{(1)} - x^{(0)}\| &= \sqrt{(2.2)^2 + (1.333)^2 + (2)^2} = \sqrt{4.84 + 1.7768 + 4} = \sqrt{10.6168} \\ &= 3.258343137\end{aligned}$$

$$\begin{aligned}\|x^{(2)} - x^{(1)}\| &= \sqrt{(0.133)^2 + (2.066)^2 + (0.767)^2} = \sqrt{0.0176 + 4.2683 + 0.5882} = \sqrt{4.8741} \\ &= 2.207736397\end{aligned}$$

$$\begin{aligned}\|x^{(3)} - x^{(2)}\| &= \sqrt{(0.6728)^2 + (-0.467)^2 + (0.5828)^2} = \sqrt{0.4526 + 0.2180 + 0.3396} \\ &= \sqrt{1.0102} = 1.005087061\end{aligned}$$

$$\begin{aligned}\|x^{(4)} - x^{(3)}\| &= \sqrt{(-0.3028)^2 + (-0.6124)^2 + (0.4532)^2} = \sqrt{0.0916 + 0.3750 + 0.2053} \\ &= \sqrt{0.6719} = 0.819695065\end{aligned}$$

$$\begin{aligned}\|x^{(5)} - x^{(4)}\| &= \sqrt{(-0.1908)^2 + (0.403)^2 + (0.0017)^2} = \sqrt{0.0364 + 0.1624 + 0.00002} \\ &= \sqrt{0.19882} = 0.445892363\end{aligned}$$

Метод на Гаус-Зайдел:

$$\begin{aligned}\|x^{(1)} - x^{(0)}\| &= \sqrt{(2.2)^2 + (0.6)^2 + (-1.05)^2} = \sqrt{4.84 + 0.36 + 1.1025} = \sqrt{6.3025} \\ &= 2.510478042\end{aligned}$$

$$\begin{aligned}\|x^{(2)} - x^{(1)}\| &= \sqrt{(0.03)^2 + (-0.71)^2 + (0.1925)^2} = \sqrt{0.0009 + 0.5041 + 0.0370} = \sqrt{0.542} \\ &= 0.736206492\end{aligned}$$

$$\begin{aligned}\|x^{(3)} - x^{(2)}\| &= \sqrt{(-0.2455)^2 + (0.21)^2 + (-0.1745)^2} = \sqrt{0.0602 + 0.0441 + 0.0304} \\ &= \sqrt{0.1347} = 0.367014986\end{aligned}$$

$$\begin{aligned}\|x^{(4)} - x^{(3)}\| &= \sqrt{(0.0491)^2 + (-0.1325)^2 + (0.057)^2} = \sqrt{0.00241 + 0.0175 + 0.00324} \\ &= \sqrt{0.02315} = 0.15215124\end{aligned}$$

$$\begin{aligned}\|x^{(5)} - x^{(4)}\| &= \sqrt{(-0.0416)^2 + (0.0518)^2 + (-0.033)^2} = \sqrt{0.0017 + 0.0026 + 0.0009} \\ &= \sqrt{0.0052} = 0.072111025\end{aligned}$$

Вредностите покажуваат дека разликата помеѓу последователните итерации постепено се намалува, што укажува дека и двата метода конвергираат.

Табела за норма на разлика помеѓу итерации		
Итерација k	Јакоби	Гаус-Зайдел
0	0	0
1	3.258343137	2.510478042
2	2.207736397	0.736206492
3	1.005087061	0.367014986
4	0.819695065	0.15215124
5	0.445892363	0.072111025

19. Конвергенција помеѓу последователни итерации

7.7. Анализа на грешката во однос на точното решение

Во анализата на итеративните методи потребно е да постои референтно решение со кое ќе се споредуваат добиените апроксимации. За таа цел системот од линеарни равенки најпрво се решава со Гаусова елиминација, која претставува директен метод за решавање на системи од линеарни равенки.

Добиеното решение $x = (-2, 0, 1)$ се смета за точно решение, со кое понатаму се споредуваат резултатите добиени со итеративните методи.

Покрај анализата на разликата помеѓу последователните итерации, можно е директно да се измери и грешката во однос на точното решение.

За таа цел се дефинира вектор на грешка:

$$e^{(k)} = x - x^{(k)}$$

каде:

- x е точното решение добиено со Гаусова елиминација;
- $x^{(k)}$ е апроксимацијата добиена во k-тата итерација.

Големината на грешката се пресметува преку Евклидовата норма:

$$\|e^{(k)}\|_2 = \|x - x^{(k)}\|_2$$

Колку што оваа вредност е помала, толку апроксимацијата е поблиску до точното решение.

Метод на Јакоби:

$$\|x - x^{(1)}\| = \sqrt{(0.2)^2 + (1.333)^2 + (-1)^2} = \sqrt{0.04 + 1.7768 + 1} = \sqrt{2.8168} = 1.78332506$$

$$\begin{aligned}\|x - x^{(2)}\| &= \sqrt{(0.333)^2 + (-0.733)^2 + (0.233)^2} = \sqrt{0.1108 + 0.5372 + 0.0542} = \sqrt{0.7022} \\ &= 0.837973746\end{aligned}$$

$$\begin{aligned}\|x - x^{(3)}\| &= \sqrt{(-0.3398)^2 + (-0.266)^2 + (0.3498)^2} = \sqrt{0.1352 + 0.0707 + 0.1223} \\ &= \sqrt{0.3282} = 0.572887423\end{aligned}$$

$$\begin{aligned}\|x - x^{(4)}\| &= \sqrt{(-0.037)^2 + (0.3464)^2 + (0.1034)^2} = \sqrt{0.001369 + 0.11999 + 0.0106} \\ &= \sqrt{0.131959} = 0.363261613\end{aligned}$$

$$\begin{aligned}\|x - x^{(5)}\| &= \sqrt{(0.1538)^2 + (-0.0566)^2 + (0.1051)^2} = \sqrt{0.0236 + 0.0032 + 0.0110} \\ &= \sqrt{0.0378} = 0.194422221\end{aligned}$$

Метод на Гаус-Зайдел:

$$\begin{aligned}\|x - x^{(1)}\| &= \sqrt{(0.2)^2 + (0.6)^2 + (0.05)^2} = \sqrt{0.04 + 0.36 + 0.0025} = \sqrt{0.4025} \\ &= 0.634428877\end{aligned}$$

$$\begin{aligned}\|x - x^{(2)}\| &= \sqrt{(0.23)^2 + (0.11)^2 + (0.1425)^2} = \sqrt{0.0529 + 0.0121 + 0.02030} = \sqrt{0.0863} \\ &= 0.293768616\end{aligned}$$

$$\begin{aligned}\|x - x^{(3)}\| &= \sqrt{(-0.0155)^2 + (0.100)^2 + (0.32)^2} = \sqrt{0.00023 + 0.01 + 0.1024} = \sqrt{0.11263} \\ &= 0.335603933\end{aligned}$$

$$\begin{aligned}\|x - x^{(4)}\| &= \sqrt{(0.0336)^2 + (-0.0325)^2 + (0.025)^2} = \sqrt{0.00112 + 0.00105 + 0.000625} \\ &= \sqrt{0.002795} = 0.052867759\end{aligned}$$

$$\begin{aligned}\|x - x^{(5)}\| &= \sqrt{(-0.008)^2 + (-0.0193)^2 + (0.008)^2} = \sqrt{0.000064 + 0.00037 + 0.000064} \\ &= \sqrt{0.000498} = 0.022315913\end{aligned}$$

Резултатите покажуваат дека и двата метода постепено конвергираат кон решението на системот. Сепак, од вредностите на нормите може да се забележи дека методот на Гаус-Зајдел достигнува помала грешка за помал број итерации. Ова се должи на фактот што во методот на Гаус-Зајдел новодобиените вредности се користат веднаш во истата итерација, што овозможува побрза конвергенција во однос на методот на Јакоби.

Табела за грешка со Гаусово решение		
Итерација k	Јакоби	Гаус-Зајдел
0	2.236067977	2.236067977
1	1.678332506	0.634428877
2	0.837973746	0.293768616
3	0.572887423	0.335603933
4	0.363261613	0.052867759
5	0.194422221	0.022315913

20. Конвергенција во однос на точното решение добиено со методот на Гаусова елиминација

7.8. Графичка анализа на конвергенцијата на решенијата

На графикот, прикажан во продолжение е прикажана промената на вредностите на непознатите x_1 , x_2 и x_3 низ последователните итерации при примена на методот на Јакоби и методот на Гаус-Зајдел. На хоризонталната оска е претставен бројот на итерации, додека на вертикалната оска се прикажани вредностите на непознатите во секоја итерација.

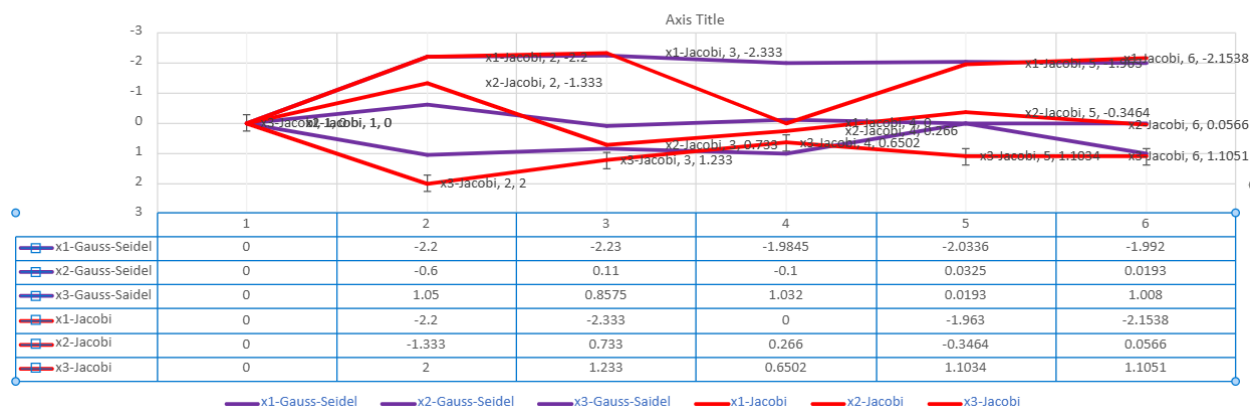
Од графикот може да се забележи дека вредностите на непознатите постепено се стабилизираат и се приближуваат кон точното решение на системот, кое изнесува $x = (-2, 0, 1)$.

Кај методот на Јакоби (прикажан со црвени линии) промените на вредностите меѓу итерациите се поголеми, особено во првите неколку чекори. Вредностите на непознатите осцилираат околу точното решение пред постепено да се стабилизираат. Ова е карактеристично за методот на Јакоби, бидејќи во секоја итерација новите вредности се пресметуваат исклучиво врз основа на вредностите од претходната итерација.

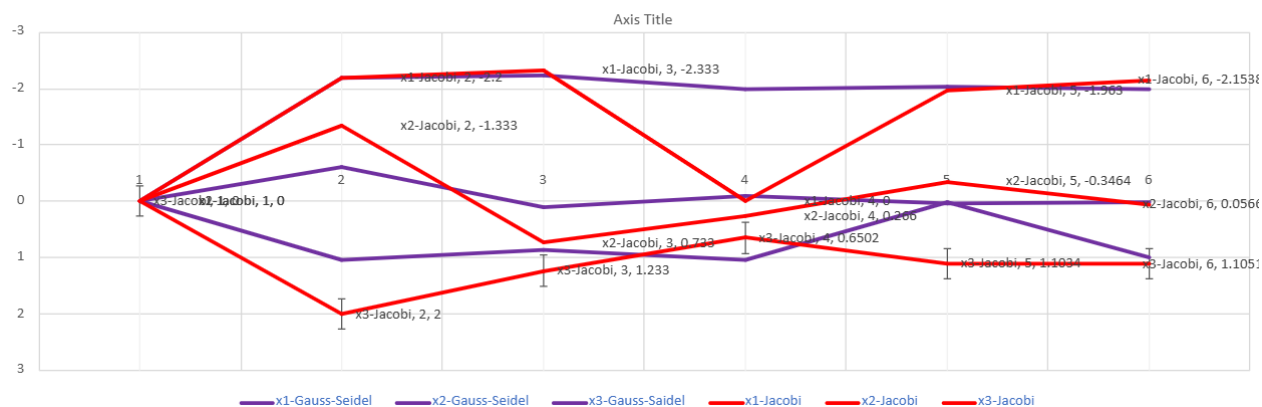
Од друга страна, методот на Гаус-Зајдел (прикажан со виолетови линии) може да се забележи побрзо приближување кон точното решение. Вредностите на непознатите се стабилизираат со помали осцилации и со помал број итерации. Причината за ова е што во методот на Гаус-Зајдел новодобиените вредности на непознатите веднаш се користат при пресметување на следните вредности во истата итерација.

Графичкиот приказ јасно ја потврдува теоретската карактеристика на овие два итеративни метода – методот на Гаус-Зајдел обично конвергира побрзо од методот на Јакоби. Со зголемување на бројот на итерации, вредностите на сите непознати се приближуваат кон стабилни вредности кои се совпаѓаат со точното решение добиено со директниот метод на Гаусова елиминација.

Конвергенција на решенијата меѓу итерации на методот на Јакоби и методот на Гаус -Зајдел



Конвергенција на решенијата меѓу итерации на методот на Јакоби и методот на Гаус -Зајдел



8. Практична верификација на програмската имплементација

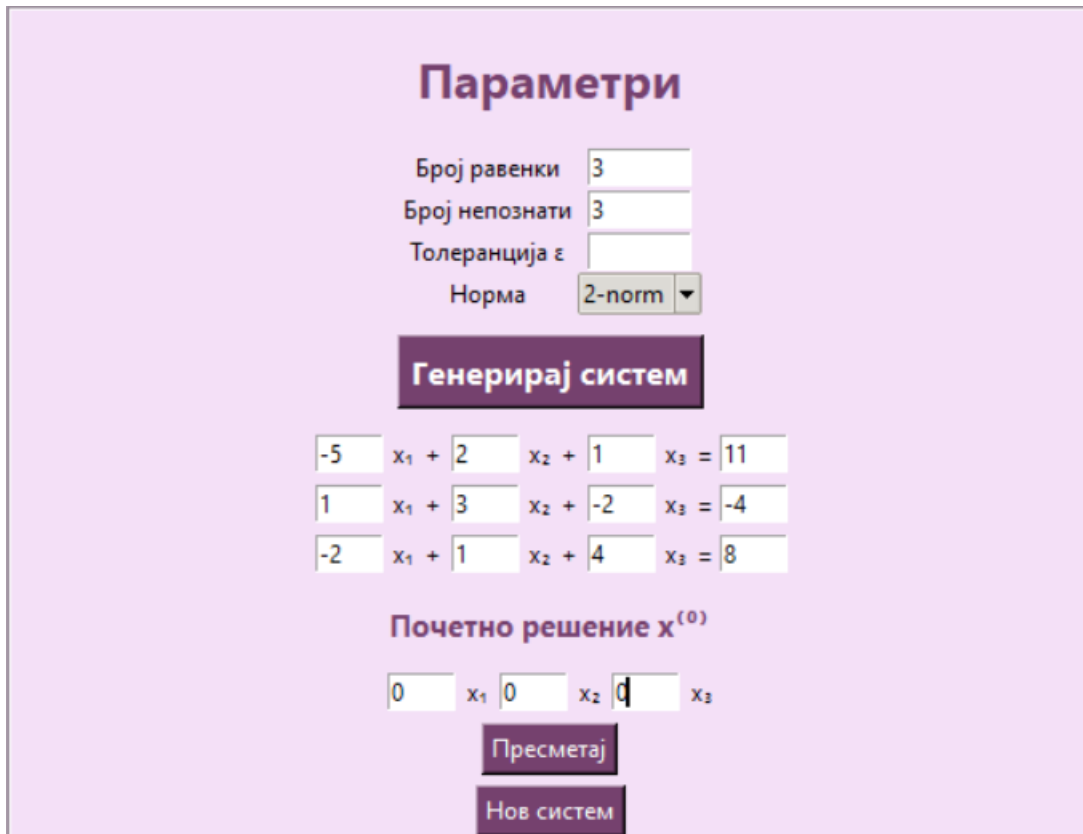
Во ова поглавје се врши практична проверка на точноста на развиената програмска апликација. Со цел да се валидира имплементацијата на нумеричките алгоритми, истиот тест пример што беше рачно решен во претходното поглавје се внесува во програмата и се анализираат добиените резултати.

Се разгледува истиот систем од три линеарни равенки со три непознати:

$$\begin{cases} -5x_1 + 2x_2 + x_3 = 11 \\ x_1 + 3x_2 - 2x_3 = -4 \\ -2x_1 + x_2 + 4x_3 = 8 \end{cases}$$

Во програмската имплементација се внесуваат коефициентите на матрицата A , векторот на слободни членови b , како и почетната апроксимација:

$$x^{(0)} = (0, 0, 0)$$



Параметри

Број равенки

Број непознати

Толеранција ϵ

Норма

Генерирај систем

-5	x_1	2	x_2	1	x_3	=	11
1	x_1	3	x_2	-2	x_3	=	-4
-2	x_1	1	x_2	4	x_3	=	8

Почетно решение $x^{(0)}$

x_1 x_2 x_3

Пресметај

Нов систем

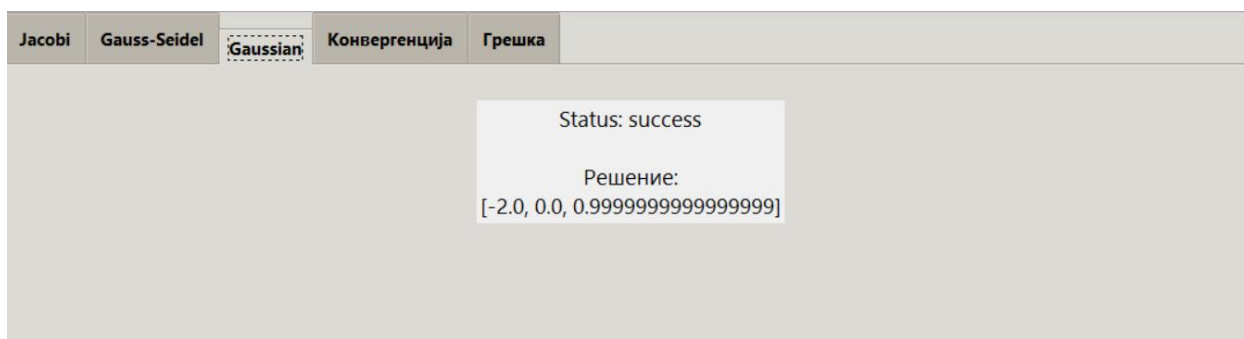
21. Внес на параметрите и коефициентите на системот линеарни равенки

Програмата го решава системот со примена на следните нумерички методи:

- Гаусова елиминација (директен метод);
- Метод на Јакоби;
- Метод на Гаус-Зајдел.

По извршувањето на пресметките, програмата ги прикажува добиените резултати во табеларна форма, при што се прикажуваат вредностите на непознатите во секоја итерација. Дополнително, се генерира и графички приказ на конвергенцијата на решението, што овозможува визуелна анализа на процесот на приближување кон точното решение.

Добиеното решение со методот на Гаусова елиминација во програмата е $x = (-2, 0, 1)$.



22. Резултат од методот на Гаусова елиминација

Овој резултат се совпаѓа со решението добиено при рачното решавање на системот, што потврдува дека имплементацијата на алгоритмот е коректна.

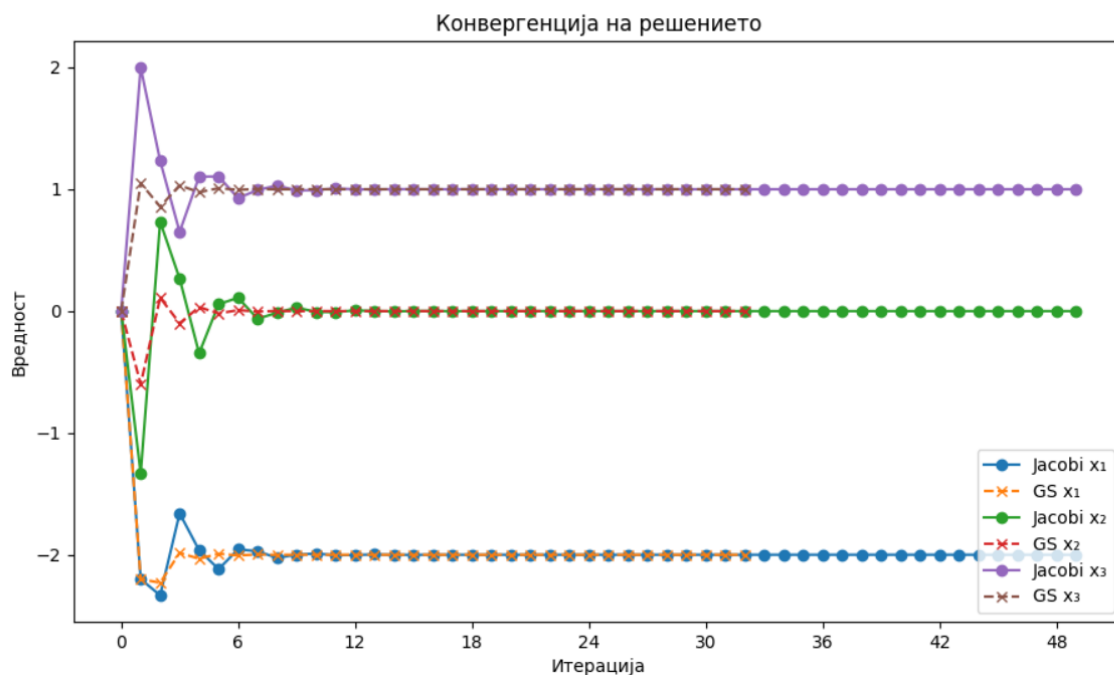
Кај итеративните методи, вредностите на непознатите постепено се приближуваат кон истото решение. Резултатите добиени од програмата се во согласност со рачно пресметаните апроксимации, со мали разлики кои се должат на заокружување на децималните вредности.

Jacobi	Gauss-Seidel	Gaussian	Конвергенција	Грешка	
k	x ₁		x ₂		error
0	0.0		0.0		0
1	-2.2		-1.3333333333333333		2.2
2	-2.3333333333333333		0.7333333333333334		2.0666666666666664
3	-1.6600000000000001		0.26666666666666633		0.6733333333333329
4	-1.9633333333333334		-0.3466666666666665		0.6133333333333328
5	-2.118		0.05666666666666664		0.4033333333333315

23. Резултат од методот на Јакоби

Jacobi	Gauss-Seidel	Gaussian	Конвергенција	Грешка	
k	x ₁		x ₂		error
0	0.0		0.0		0
1	-2.2		-0.6		2.2
2	-2.23		0.1099999999999998		0.7099999999999999
3	-1.9845		-0.1001666666666668		0.24550000000000005
4	-2.0335083333333333		0.0330305555555556		0.13319722222222224
5	-1.991790138888889		-0.019411157407407398		0.052441712962962996

24. Резултат од методот на Гаус-Зајдел



Од споредбата може да се заклучи дека програмската имплементација успешно ги реализира нумеричките методи, резултатите од програмата се совпаѓаат со рачно

добиените резултати и итеративните методи постепено конвергираат кон точното решение на системот.

9. Заклучок

Во рамките на оваа семинарска работа беше разгледано решавањето на системи од линеарни равенки со примена на различни нумерички методи. Посебен акцент беше ставен на споредбата помеѓу директниот метод на Гаусова елиминација и итеративните методи – методот на Јакоби и методот на Гаус-Зајдел.

Најпрво беше дадена теоретска основа за секој од методите, при што беа објаснети основните принципи на нивното функционирање и условите за нивна примена. Потоа беше разгледан конкретен тест пример на систем од три линеарни равенки со три непознати, кој беше решен рачно со примена на сите три методи. Рачното решавање овозможи јасно следење на чекорите на алгоритмите и анализа на процесот на конвергенција кај итеративните методи.

Во продолжение беше извршена анализа на точноста на добиените апроксимации преку пресметување на нормата на векторот на грешка, како и преку следење на разликата помеѓу последователните итерации. Резултатите покажаа дека и методот на Јакоби и методот на Гаус-Зајдел постепено конвергираат кон точното решение на системот, но дека методот на Гаус-Зајдел обезбедува побрза конвергенција и помала грешка за помал број итерации.

Дополнително, беше развиена програмска апликација која ги имплементира овие нумерички методи и овозможува автоматизирано решавање на системи од линеарни равенки. Преку практичниот тест пример беше извршена споредба помеѓу рачно добиените резултати и резултатите добиени со програмата. Анализата покажа дека програмската имплементација дава резултати кои се во согласност со рачните пресметки, со мали отстапувања кои се должат на заокружување на децималните вредности.

Врз основа на извршената анализа може да се заклучи дека имплементираните нумерички методи успешно овозможуваат решавање на системи од линеарни равенки и претставуваат ефикасна алатка во нумеричката анализа. Развиената програмска апликација ја потврдува практичната применливост на овие методи и овозможува нивна поедноставна и побрза примена при решавање на различни математички и инженерски проблеми.

10. Користена литература

„Практикум по нумерички методи“ - Доц. д-р **Јасмина Вета Буралиева**, Факултет за информатика, УГД – Штип, Доц. д-р **Васко Кокаланов**, Факултет за информатика, УГД-Штип

„Нумерички методи скрипта“ – д-р **Васко Кокаланов**, д-р **Владо Гичев**, м-р **Александра Ристеска**, Факултет за информатика, УГД-Штип

Материјали достапни на курсот за е-учење преку официјалниот веб-сајт на **Универзитетот „Гоце Делчев“ – Штип**